

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

Курсова робота

з дисципліни «Програмування»

на тему: «Система проведення тендерів»

Виконав:

студент 1 курсу, групи ІА-51

Крапивний Тарас Олегович

(підпис)

Керівник:

асистент кафедри ІСТ

Петренко Петро Петрович

(підпис)

Засвідчую, що у цій курсовій
роботі немає запозичень з праць
інших авторів без відповідних
посилань.

Студент _____

Київ – 2026 року

ЗМІСТ

ВСТУП.....	3
1 ВИМОГИ ДО СИСТЕМИ.....	5
1.1 Функціональні вимоги до системи.....	5
1.2 Нефункціональні вимоги до системи.....	6
2 СЦЕНАРІЇ ВИКОРИСТАННЯ СИСТЕМИ.....	7
2.1 Діаграма прецедентів.....	7
2.2 Опис сценаріїв використання системи.....	9
3 АРХІТЕКТУРА СИСТЕМИ.....	17
4 РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ.....	20
4.1 Загальна структура проєкту.....	20
4.2 Компоненти рівня доступу до даних.....	22
4.3 Компоненти рівня бізнес-логіки.....	24
4.4 Компоненти рівня інтерфейсу коористувача.....	26
ВИСНОВКИ.....	27
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	29
ДОДАТКИ.....	30

ВСТУП

В умовах сучасної цифрової економіки автоматизація бізнес-процесів є ключовим фактором підвищення ефективності будь-якого підприємства. Одним із найважливіших етапів комерційної діяльності є пошук підрядників та закупівля послуг або товарів. Проведення тендерів дозволяє забезпечити прозорість цього процесу, створити конкурентне середовище та обрати виконавця з найвигіднішими умовами.

Проте організація тендерів традиційними методами є вкрай неефективною, оскільки супроводжується великими витратами часу, ризиком втрати даних та складністю структурування отриманих пропозицій. Використання ж масштабних державних або міжнародних майданчиків не завжди є доцільним для внутрішніх потреб малого та середнього бізнесу через їхню перевантаженість функціоналом та складність інтеграції. Відтак виникає актуальна потреба у розробці легкої, швидкої та інтуїтивно зрозумілої інформаційної системи для локального управління тендерними закупівлями.

Метою роботи є проектування та програмна реалізація інформаційної системи для організації тендерів, збору комерційних пропозицій та зручного вибору виконавців.

Для досягнення поставленої мети у роботі вирішуються такі основні задачі:

- Проектування архітектури бази даних: створення нормалізованої реляційної структури (MySQL) для надійного зберігання інформації про користувачів, відкриті тендери та подані заявки із забезпеченням цілісності даних;

- Розробка серверної частини (Back-end): реалізація бізнес-логіки системи мовою програмування Java зі створенням RESTful API (з використанням технологій Jakarta EE / Jersey) для обробки клієнтських запитів;
- Розробка клієнтської частини (Front-end): створення інтерактивного користувацького інтерфейсу (HTML, CSS, JavaScript) для взаємодії з системою через веб-браузер;
- Реалізація алгоритмів контролю та безпеки: впровадження механізмів авторизації, розмежування прав доступу, контролю статусів життєвого циклу тендеру та захисту від дублювання заявок.

Об'єктом дослідження є процеси автоматизації проведення тендерів та збору комерційних пропозицій.

Предметом дослідження є методи та інструментальні засоби розробки веб-додатків з використанням технологій Java та реляційних баз даних.

1 ВИМОГИ ДО СИСТЕМИ

1.1 Функціональні вимоги до системи

Система має відповідати наступним функціональним вимогам:

- користувач повинен мати можливість зареєструвати новий акаунт та авторизуватися в системі для отримання доступу до функціоналу;
- авторизований користувач повинен мати можливість створювати нові тендери, вказуючи назву та опис, при цьому система автоматично фіксує його як власника та встановлює статус тендеру як активний;
- користувач повинен мати можливість управляти власними тендерами: видаляти їх, змінювати їхній статус та обирати переможця;
- користувач повинен мати можливість переглядати всі подані пропозиції до своїх тендерів для їх аналізу та вибору найкращого виконавця;
- користувач повинен мати можливість переглядати загальний список усіх активних тендерів у системі, створених іншими учасниками;
- користувач повинен мати можливість здійснювати пошук тендерів за ключовими словами у назві або описі;
- користувач повинен мати можливість подавати тендерну пропозицію до відкритих тендерів інших користувачів, вказуючи свою ціну та супровідний коментар (при цьому діє обмеження, що не можна більше однієї пропозиції від одного користувача на конкретний тендер).

1.2 Нефункціональні вимоги до системи

Система має відповідати наступним нефункціональним вимогам:

- система повинна мати відкриту клієнт-серверну архітектуру на базі REST API;
- для обміну даними між клієнтом та сервером повинен використовуватися стандартизований формат JSON;
- система повинна мати зручний та інтуїтивно-зрозумілий веб-інтерфейс для взаємодії з користувачем;
- збереження інформації повинно здійснюватися в базі даних (MySQL) із забезпеченням цілісності даних за допомогою зовнішніх ключів та унікальних індексів.

2 СЦЕНАРІЇ ВИКОРИСТАННЯ СИСТЕМИ

2.1 Діаграма прецедентів

Діаграма прецедентів системи представлена на рис. 2.1.

Основним актором системи є Авторизований користувач. У системі немає жорсткого поділу на ролі на рівні бази даних. Залежно від контексту, користувач може виступати у двох ролях:

- Власник тендеру: коли користувач створює новий тендер, він отримує права на його управління (зміна статусу, видалення, перегляд пропозицій та вибір переможця).
- Виконавець: коли користувач переглядає тендери, створені іншими учасниками системи, він має можливість подавати до них свої цінові пропозиції.

Усім користувачам доступна базова функціональність: авторизація, перегляд списку відкритих тендерів та їх пошук за ключовими словами у назві або описі. Детально основні сценарії використання описані у наступному підрозділі.

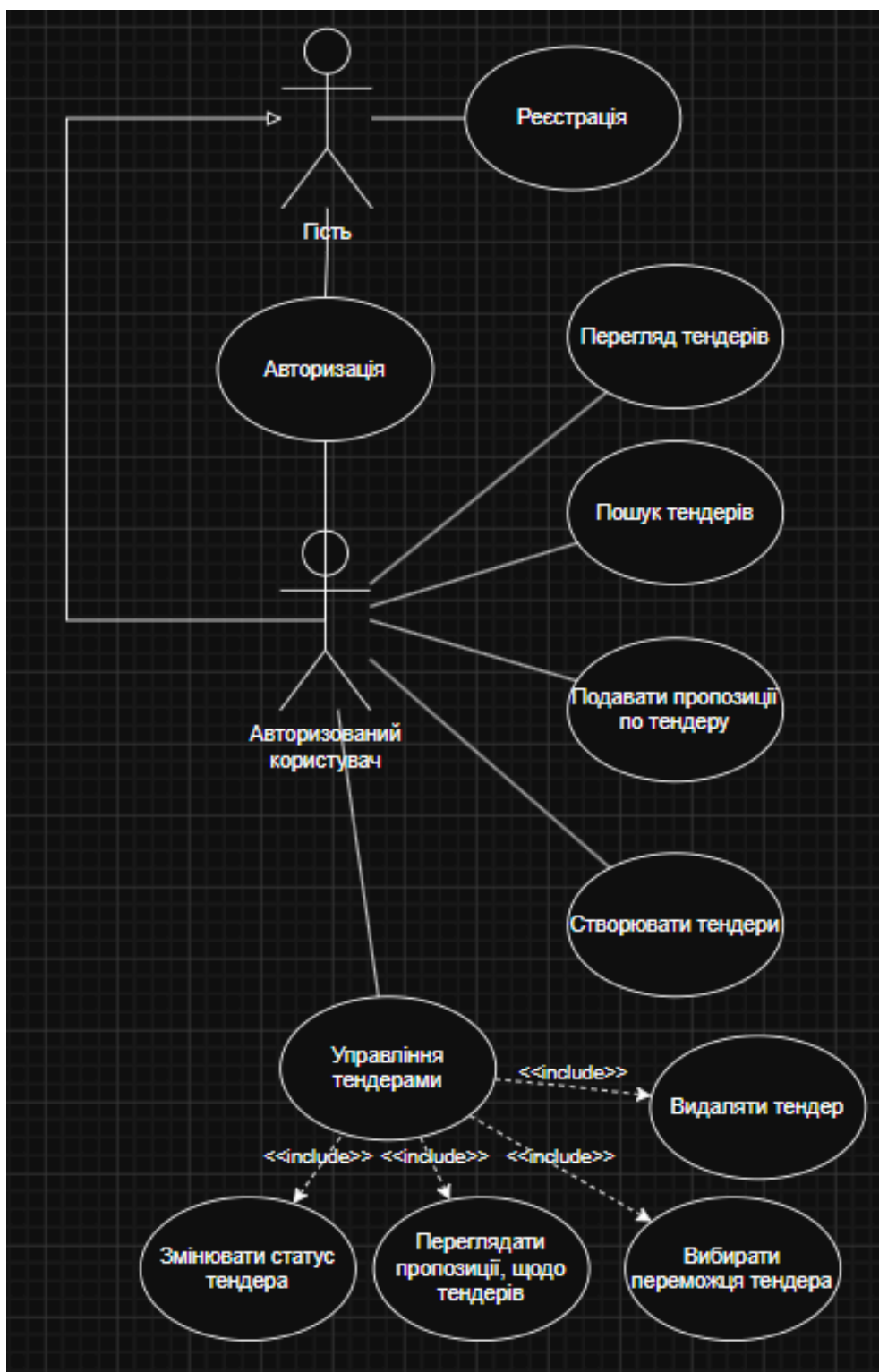


Рисунок 2.1 – Діаграма прецедентів

2.2 Опис сценаріїв використання системи

Детальні описи сценаріїв використання наведено у таблицях 2.1-2.4.

Таблиця 2.1 – Сценарій використання «Перегляд списку активних тендерів»

Назва	Перегляд активних тендерів
ID	1
Опис	Користувач переглядає всі доступні тендери
Актори	Авторизований користувач
Вигода компанії	Забезпечення прозорості та швидкого доступу учасників до актуальних тендерів
Частота користування	Постійно
Тригери	Користувач відкриває головну сторінку системи
Передумови	Сервер працює, налаштовано з'єднання з базою даних MySQL. Користувач авторизований.
Постумови	Користувач бачить на екрані сформований список тендерів.
Основний розвиток	Користувач переходить на головну сторінку. Клієнтська частина надсилає GET-запит. Сервер виконує SQL-запит до бази даних, дістає всі активні тендери та повертає їх у форматі JSON. Фронтенд створює інтерфейс.
Альтернативний розв'язок	У системі ще не створено жодного тендеру. Клієнт отримує порожній масив та виводить відповідне повідомлення.
Виняткові ситуації	—

В таблиці 2.2 представлений сценарій використання «Створення нового тендеру».

Таблиця 2.2 – Сценарій використання «Створення нового тендеру»

Назва	Створення нового тендеру
ID	2
Опис	Користувач створює новий тендер, вказуючи його назву та детальний опис
Актори	Авторизований користувач (стає власником)
Вигода компанії	Наповнення платформи новими замовленнями для подальшого залучення виконавців
Частота користування	Часто
Тригери	Відправка форми на сторінці створення тендеру.
Передумови	Користувач авторизований у системі.
Постумови	Новий тендер зберігається у базі даних і стає доступним для перегляду іншим користувачам
Основний розвиток	Користувач заповнює форму. ID власника додається автоматично з локального сховища. Дані відправляються POST-запитом. Сервер робить запис у таблицю tenders зі статусом ACTIVE.
Альтернативний розв'язок	–
Виняткові ситуації	Відправлено порожні дані або втрачено зв'язок з БД. Сервер повертає код помилки.

В таблиці 2.3 представлений сценарій використання «Подача тендерної пропозиції».

Таблиця 2.3 – Сценарій використання «Подача тендерної пропозиції»

Назва	Подача тендерної пропозиції
ID	3
Опис	Користувач пропонує свої послуги для виконання конкретного тендеру, вказуючи ціну та коментар
Актори	Авторизований користувач
Вигода компанії	Створення конкуренції між виконавцями для вибору найкращої пропозиції
Частота користування	Часто
Тригери	Натискання кнопки “Відправити пропозицію” на сторінці тендеру.
Передумови	Тендер має статус ACTIVE. Користувач не є власником цього тендеру і ще не подавав заявку на нього.
Постумови	Пропозиція записується в базу даних із прив'язкою до тендеру.
Основний розвиток	Користувач вводить ціну та опис. Сервер перевіряє статус тендеру та перевіряє БД на відсутність дублікатів від цього користувача. Якщо перевірки пройдені, створюється запис у таблиці tender_proposals.
Альтернативний розв'язок	—
Виняткові ситуації	Тендер вже закритий. Спроба подати другу пропозицію (користувач отримує повідомлення про помилку).

В таблиці 2.4 представлений сценарій використання «Перегляд пропозицій, щодо тендерів».

Таблиця 2.4 – Сценарій використання «Перегляд пропозицій, щодо тендерів»

Назва	Перегляд пропозицій, щодо тендерів
ID	4
Опис	Власник тендеру переглядає всі відгуки та цінові пропозиції, залишені іншими виконавцями.
Актори	Авторизований користувач (власник тендеру)
Вигода компанії	Надання замовнику зручного інструменту для аналізу ринку та порівняння кандидатів.
Частота користування	Часто
Тригери	Перехід на сторінку управління власним тендером.
Передумови	Користувач є власником даного тендеру.
Постумови	Замовник бачить масив усіх пропозицій.
Основний розвиток	Клієнтська частина надсилає GET-запит на отримання пропозицій за конкретним ID тендеру. Сервіс фільтрує загальний список пропозицій з бази даних. Список повертається власнику та створюється в інтерфейсі.
Альтернативний розв'язок	До тендеру ще ніхто не подав пропозицію. Сервер повертає порожній список, інтерфейс виводить повідомлення “Пропозицій ще немає”.
Виняткові ситуації	–

В таблиці 2.5 представлений сценарій використання «Пошук тендерів».

Таблиця 2.5 – Сценарій використання «Пошук тендерів»

Назва	Пошук тендерів
ID	5
Опис	Користувач здійснює пошук серед активних тендерів, використовуючи ключові слова для фільтрації за назвою або описом.
Актори	Авторизований користувач
Вигода компанії	Забезпечення зручної навігації платформою, економія часу виконавця на пошук релевантних замовлень.
Частота користування	Часто
Тригери	Введення тексту у рядок пошуку та натискання кнопки “Пошук”.
Передумови	Користувач авторизований, у базі даних існують відкриті тендери.
Постумови	На екрані відображається відфільтрований перелік тендерів.
Основний розвиток	Клієнтська частина відправляє GET-запит із параметром пошуку на сервер. Сервер виконує SQL-запит, дістає відповідні записи та повертає їх у форматі JSON. Фронтенд динамічно оновлює список на сторінці.
Альтернативний розв'язок	За вказаним ключовим словом збігів не знайдено. Клієнт отримує порожній масив та виводить повідомлення “За вашим запитом тендерів не знайдено”.
Виняткові ситуації	—

В таблиці 2.6 представлений сценарій використання «Видалення власного тендеру».

Таблиця 2.6 – Сценарій використання «Видалення власного тендеру»

Назва	Видалення власного тендеру
ID	6
Опис	Власник тендеру безповоротно видаляє свій тендер із системи
Актори	Авторизований користувач (власник тендеру)
Вигода компанії	Підтримання актуальності бази даних, можливість для замовника скасувати неактуальне замовлення.
Частота користування	Рідко
Тригери	Натискання кнопки “Видалити тендер” на панелі управління власним тендером.
Передумови	Користувач є власником обраного тендеру.
Постумови	Запис про тендер та всі пов'язані з ним пропозиції назавжди видаляються з бази даних MySQL.
Основний розвиток	Користувач підтверджує намір видалити тендер. Клієнт відправляє DELETE-запит. Сервер перевіряє права доступу (порівнює ID користувача та ID власника тендеру в базі). У разі збігу сервер виконує SQL-запит на видалення тендеру та його пропозицій, після чого повертає статус успіху. Користувача перенаправляє на головну сторінку.
Альтернативний розв'язок	–
Виняткові ситуації	Спроба видалити чужий тендер в обхід інтерфейсу. Сервер блокує дію та повертає помилку доступу (403 Forbidden).

В таблиці 2.7 представлений сценарій використання «Реєстрація та авторизація».

Таблиця 2.7 – Сценарій використання «Реєстрація та авторизація»

Назва	Реєстрація та авторизація
ID	7
Опис	Гість створює новий обліковий запис або виконує вхід до існуючого для отримання доступу до основного функціоналу.
Актори	Гість
Вигода компанії	Ідентифікація учасників платформи, персоналізація дій, захист системи від несанкціонованого доступу.
Частота користування	Постійно
Тригери	Відправка форми реєстрації або авторизації на відповідній веб-сторінці.
Передумови	Сервер запущено, підключення до бази даних активне.
Постумови	Користувач набуває статусу “Авторизований”, його сесійні дані зберігаються на стороні клієнта.
Основний розвиток	Гість вводить облікові дані та натискає “Увійти”. Клієнт формує POST-запит. Сервер перевіряє наявність запису в базі даних. У разі успіху повертає об’єкт користувача. Фронтенд зберігає дані у localStorage та перенаправляє користувача на головну сторінку системи.
Альтернативний розв'язок	Користувач немає облікового запису та переходить до реєстрації. Гість натискає кнопку “Зареєструватися” та перенаправляється на сторінку реєстрації. Гість вводить новий унікальний логін та пароль, після чого натискає кнопку створення акаунту. Клієнт відправляє POST-запит на сервер. Сервер успішно записує нового користувача в БД та повертає статус успіху.
Виняткові ситуації	Введено невірний логін або пароль при вході. Спроба зареєструватися під логіном, що вже існує. Сервер повертає повідомлення про помилку, яке відображається користувачу.

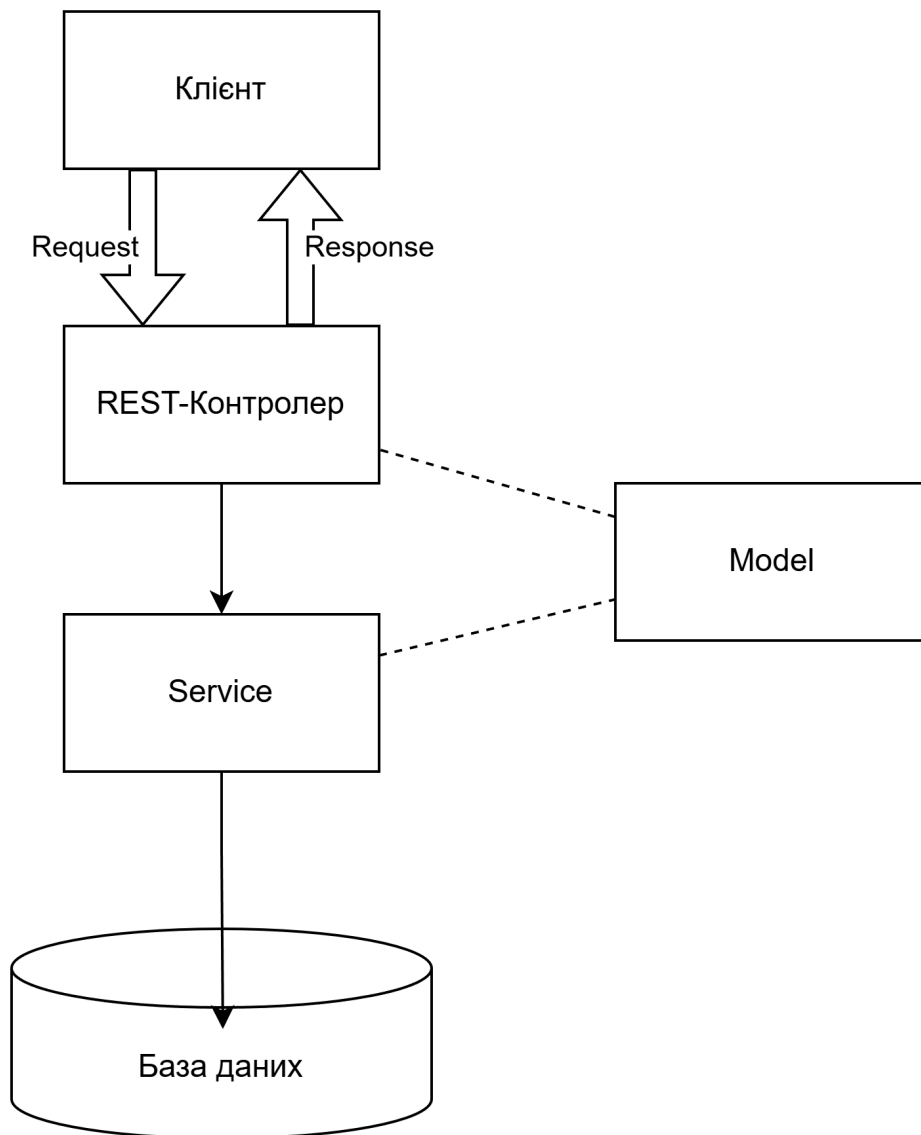
В таблиці 2.8 представлений сценарій використання «Вибір переможця».

Таблиця 2.8 – Сценарій використання «Вибір переможця»

Назва	Вибір переможця
ID	8
Опис	Власник тендеру аналізує подані пропозиції, обирає найкращу з них, тим самим закриваючи тендер.
Актори	Авторизований користувач (власник тендеру)
Вигода компанії	Успішне завершення бізнес-циклу, фіксація домовленості між Замовником та Виконавцем.
Частота користування	Часто
Тригери	Натискання кнопки “Обрати переможця” в панелі управління.
Передумови	Тендер перебуває у статусі ACTIVE, до нього подано хоча б одну пропозицію.
Постумови	Статус тендеру в базі даних змінюється на CLOSED. Функція подачі нових пропозицій для інших користувачів блокується.
Основний розвиток	Власник приймає рішення та ініціює зміну статусу. Клієнт надсилає запит на сервер із зазначенням ID тендеру, ID обраного виконавця. Сервер виконує SQL-запит на оновлення запису в базі даних, фіксуючи новий статус.
Альтернативний розв'язок	—
Виняткові ситуації	Тендер вже був закритий раніше. Сервер відхиляє запит і не вносить змін.

3 АРХІТЕКТУРА СИСТЕМИ

Загальна архітектура системи наведена на рис. 3.1.



Система складається з наступних елементів:

- графічний інтерфейс;
- серверна частина;
- база даних.

Розроблена інформаційна система проведення тендерів побудована на основі класичної тривірневої клієнт-серверної архітектури. Цей підхід

забезпечує чіткий розподіл обов'язків між компонентами, полегшує масштабування системи, її тестування та подальшу підтримку.

Архітектура системи складається з наступних взаємопов'язаних рівнів:

- Клієнтський рівень (Front-end): Відповідає за взаємодію з користувачем. Реалізований у вигляді багатосторінкового веб-додатку за допомогою технологій HTML, CSS та JavaScript. Цей рівень не містить складної бізнес-логіки чи прямого доступу до бази даних, а лише відмальовує інтерфейс та надсилає асинхронні HTTP-запити до сервера.

- Серверний рівень (Back-end): Є ядром системи. Розроблений мовою програмування Java з використанням специфікації Jakarta RESTful Web Services (JAX-RS / Jersey). Серверний рівень приймає запити від клієнта, виконує необхідну логіку (перевірку прав доступу, валідацію даних, перевірку унікальності), ініціює звернення до бази даних і формує відповідь.

- Рівень збереження даних (Database Layer): Відповідає за постійне та надійне зберігання інформації. Реалізований на базі реляційної системи керування базами даних MySQL. Взаємодія між серверним рівнем та базою даних здійснюється через стандартний драйвер JDBC (Java Database Connectivity).

Обмін даними між клієнтською та серверною частинами відбувається за архітектурним стилем REST (Representational State Transfer). Для передачі інформації використовується легкий та стандартизований формат обміну даними - JSON. Кожен бізнес-об'єкт системи має свій виділений контролер (Resource), який обробляє запити за відповідними маршрутами (endpoints).

Такий підхід робить систему “слабкозв'язаною” (loosely coupled) - клієнтський додаток нічого не знає про внутрішню реалізацію бази даних чи серверного коду, він лише споживає наданий сервером API.

В основі рівня збереження даних лежить нормалізована реляційна модель. База даних складається з трьох основних таблиць, об'єднаних зовнішніми ключами, що гарантує цілісність даних:

- users - таблиця для зберігання облікових даних користувачів.
- tenders - таблиця для зберігання інформації про створені замовлення. Включає назву, опис, статус (ACTIVE або CLOSED), а також зовнішній ключ ownerId, який вказує на ідентифікатор користувача, що створив тендер.
- tender_proposals - таблиця-зв'язка, що зберігає відгуки користувачів на тендери. Включає цінову пропозицію, коментар, а також два зовнішні ключі: tenderId (ідентифікатор тендеру) та executorId (ідентифікатор виконавця).

Для забезпечення коректної бізнес-логіки реалізовано структурне обмеження — складений унікальний індекс (UNIQUE CONSTRAINT) для полів tenderId та executorId у таблиці пропозицій. Це гарантує, що система відхилить будь-яку спробу користувача подати більше однієї заявки на один і той самий тендер. Крім того, налаштовано каскадне видалення, що дозволяє автоматично очищати всі пов'язані пропозиції при видаленні батьківського тендеру власником.

4 РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ

4.1 Загальна структура проекту

Загальна структура проекту представлена на рисунках 4.1.1

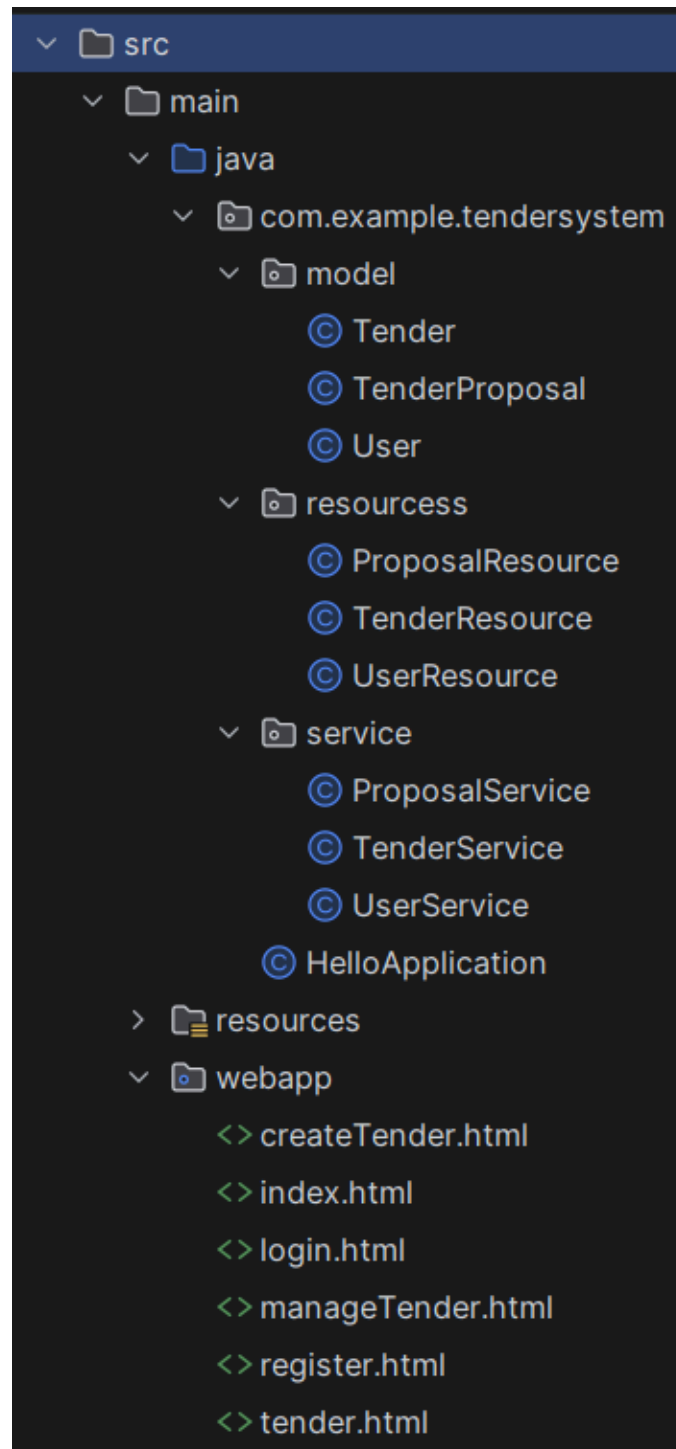


Рисунок 4.1.1 – Загальна структура проекту

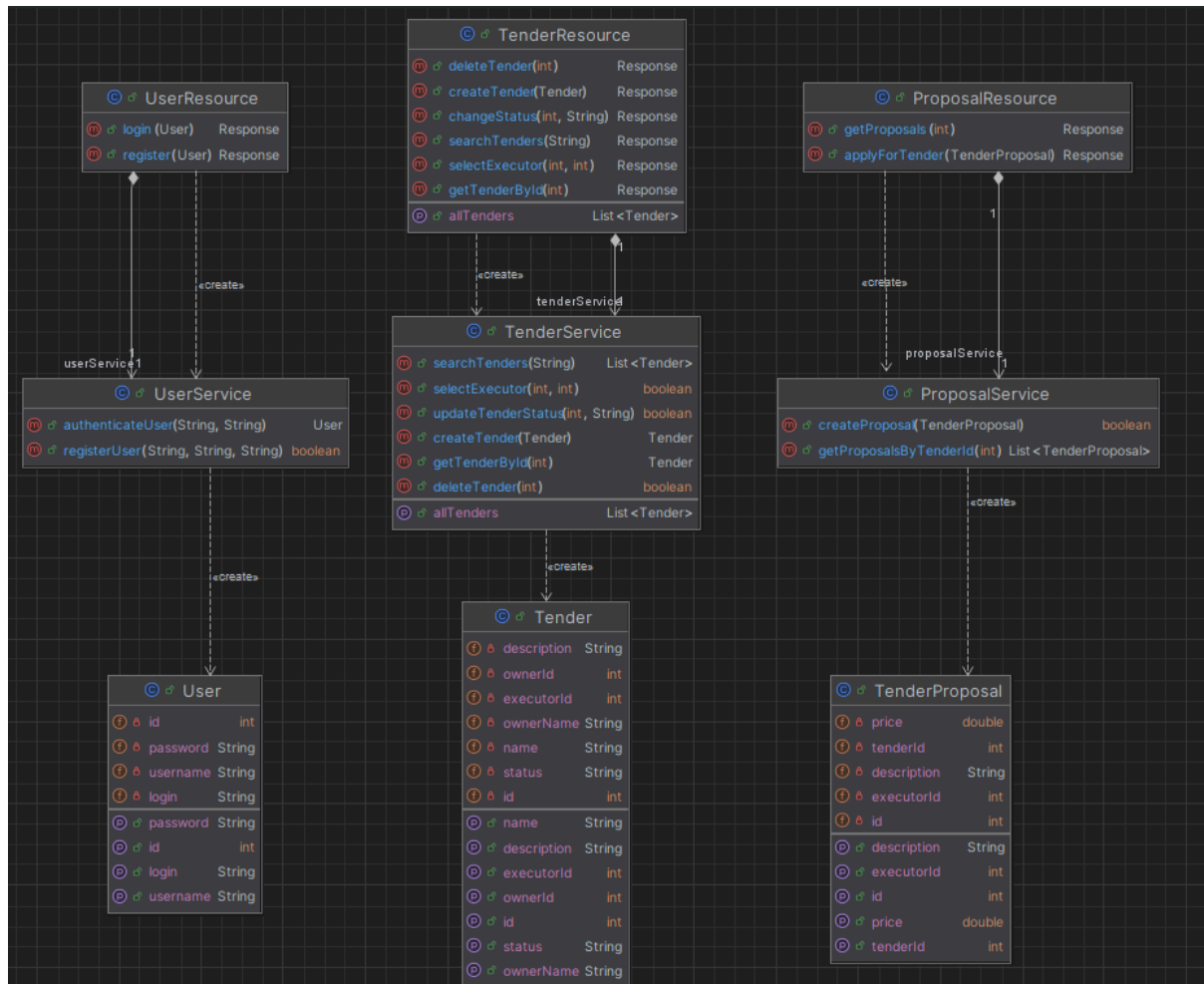


Рисунок 4.1.2 – Загальна структура проекту

Програмна реалізація серверної частини інформаційної системи виконана у середовищі розробки IntelliJ IDEA. Для управління залежностями та автоматизації процесу збірки використано інструмент Apache Maven. Структура директорій проекту відповідає стандартному патерну розробки веб-додатків на Java.

4.2 Компоненти рівня доступу до даних

Основні сутності та інтерфейси рівня доступу до даних наведені на рис. 4.2

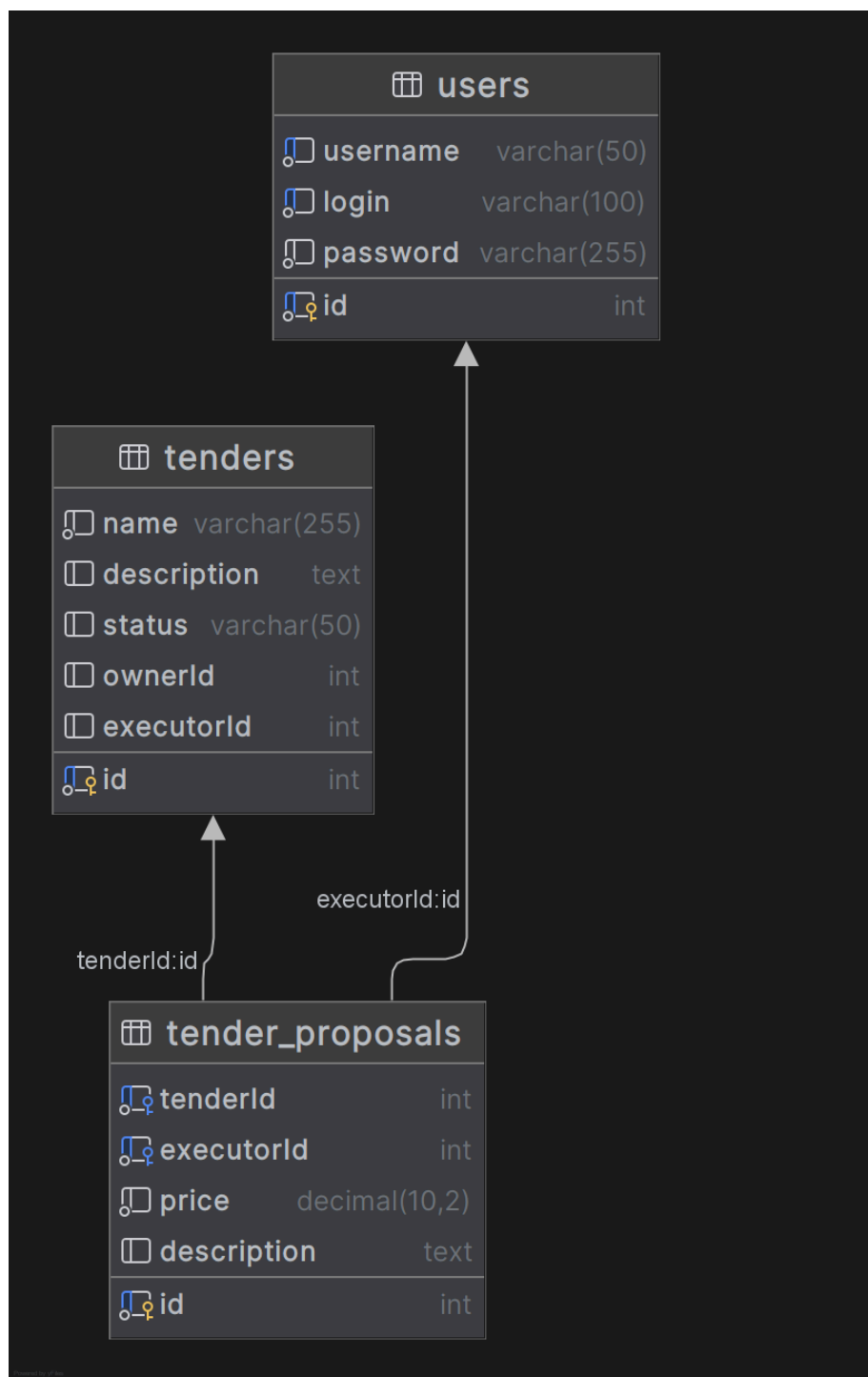


Рисунок 4.2 – Основні сутності та інтерфейси рівня доступу до даних
Далі детальний опис сутностей та інтерфейсів:

Клас User представляє модель користувача й відповідає за збереження інформації про облікові записи. Він має такі атрибути:

- id: унікальний числовий ідентифікатор користувача (int)
- login: логін користувача (String), повинен бути унікальним і не порожнім
- username: ім'я користувача для відображення в системі (String)
- password: пароль користувача (String)

Клас Tender описує замовлення й служить для групування поданих пропозицій. Він має такі атрибути:

- id: унікальний числовий ідентифікатор тендеру (int)
- name: назва тендеру (String), має бути зрозумілою та описовою
- description: детальний опис умов тендеру (String)
- status: поточний стан тендеру (String), наприклад "ACTIVE" або "CLOSED"
- ownerId: ідентифікатор користувача-власника (int)
- executorId: ідентифікатор користувача-переможця (int)
- ownerName: ім'я власника тендеру для зручного відображення (String)

Клас TenderProposal описує тендерну пропозицію (відгук) від виконавця. Він має такі атрибути:

- id: унікальний числовий ідентифікатор пропозиції (int)
- tenderId: ідентифікатор тендеру, до якого подається пропозиція (int)
- executorId: ідентифікатор виконавця (int)
- price: запропонована ціна за виконання роботи (double)
- description: супровідний коментар від виконавця (String)

Зв'язки між сутностями:

- Кожен User може бути автором кількох Tender (зв'язок один-до-багатьох).
- Кожен Tender містить багато TenderProposal (зв'язок один-до-багатьох).
- TenderProposal належить до одного User (поле executorId) і одного Tender (поле tenderId).

Використання:

Контролери приймають HTTP-запити, звертаються до моделей через сервіси та JDBC для отримання чи збереження даних.

Фронтенд (JavaScript Fetch API) заповнює і передає дані цих моделей у форматі JSON.

Шаблони HTML відображають атрибути цих моделей: список активних тендерів (index.html), деталізація тендеру (tender.html), форми створення тендерів та пропозицій.

Таким чином, моделі User, Tender і TenderProposal є основою рівня доступу до даних, забезпечуючи зберігання, валідацію та зв'язки всіх основних сутностей системи.

4.3 Компоненти рівня бізнес-логіки

Рівень бізнес-логіки та обробки запитів (Back-end) побудований за принципами архітектурного стилю REST API на основі специфікації Jakarta RESTful Web Services (JAX-RS). Головне завдання цього рівня – прийом HTTP-запитів від клієнтської частини, їх маршрутизація, валідація вхідних даних, виконання бізнес-правил системи та взаємодія з базою даних через компоненти доступу до даних.

Головною точкою входу в API є клас HelloApplication, який успадковує базовий клас Application. Цей клас ініціалізує JAX-RS контекст у контейнері сервлетів і визначає базовий URL-шлях для всіх кінцевих точок системи проведення тендерів.

Маршрутизація та обробка HTTP-запитів розподілена між класами-контролерами, які використовують специфічні анотації Jakarta для декларативного опису поведінки:

Клас `UserResource` – відповідає за автентифікацію та реєстрацію користувачів. Містить кінцеві точки для обробки запитів створення облікового запису та перевірки логіну й паролю.

Клас `TenderResource` – керує життєвим циклом тендерів. Обробляє запити на отримання списку всіх активних замовлень, створення нового тендеру автором, а також закриття або видалення тендеру його власником.

Клас `ProposalResource` – відповідає за операції, пов'язані з тендерними пропозиціями від виконавців.

Кожен із цих контролерів використовує анотацію `@Path` для визначення конкретного URL-ресурсу, анотації `@Consumes(MediaType.APPLICATION_JSON)` та `@Produces(MediaType.APPLICATION_JSON)` для автоматичної серіалізації об'єктів у формат JSON, а також анотації конкретних HTTP-методів: `@GET`, `@POST`, `@DELETE`.

Безпосередня реалізація логіки та перевірка бізнес-обмежень винесена у відповідні сервісні класи: `UserService`, `TenderService` та `ProposalService`. Обробка типових сценаріїв використання на цьому рівні відбувається за такими алгоритмами:

Створення нового ресурсу: Метод у `TenderResource` приймає об'єкт моделі `Tender`, десеріалізований з JSON POST-запиту. Запит делегується до `TenderService`, який перед виконанням операції `INSERT` у базу даних перевіряє коректність заповнення полів. У разі успішної валідації викликається JDBC-рівень, а клієнту повертається об'єкт `Response` зі статусом `201 Created` та створеним об'єктом у тілі відповіді.

Обробка виняткових ситуацій та валідація: Система захищена від некоректних дій користувача на рівні бізнес-логіки. Наприклад, під час спроби подати пропозицію через `ProposalService`, сервіс спочатку перевіряє поточний статус тендеру в базі даних. Якщо тендер уже має статус "CLOSED", або якщо даний виконавець уже подавав заявку на цей тендер раніше, система блокує операцію. Замість виконання SQL-запиту на

додавання, контролер перериває обробку й повертає клієнту об'єкт Response з HTTP-статусом 400 Bad Request або 409 Conflict та текстовим описом помилки у форматі JSON. Це гарантує цілісність бізнес-логіки незалежно від стану фронтенд-частини.

Видалення або зміна статусу: При отриманні запиту DELETE або POST на зміну стану сервісний шар обов'язково перевіряє права доступу користувача (ownerId), порівнюючи його з ідентифікатором сесії, що прийшов із клієнта. Якщо користувач не є власником тендеру, система повертає статус 403 Forbidden.

Таким чином, розроблений рівень бізнес-логіки повністю ізолює роботу з базою даних від клієнтського інтерфейсу, забезпечує жорстку валідацію даних, безпеку операцій та стандартизовану відповідь сервера за допомогою HTTP-статусів.

4.4 Компоненти рівня інтерфейсу коористувача

Клієнтська частина (Front-end) інформаційної системи розроблена у вигляді набору інтерактивних веб-сторінок за допомогою стандартних технологій HTML5, CSS3 та мови програмування JavaScript.

Основні компоненти інтерфейсу користувача розподілені за функціональними веб-сторінками:

login.html та register.html – містять екранні форми для автентифікації та реєстрації користувачів. Браузерні форми збирають введені дані, валідують їх на базовому рівні (перевірка на порожні поля) та передають на сервер.

index.html (Головна панель) – центральний інтерфейс системи, що містить динамічну таблицю або перелік усіх доступних активних тендерів, рядок текстового пошуку та елементи навігації для переходу до створення нового замовлення.

tender.html та manageTender.html – сторінки детального перегляду конкретного тендеру. Перша призначена для виконавців (містить форму відправки цінової пропозиції), друга – для замовника-власника (відображає список усіх поданих пропозицій з кнопками для вибору переможця або видалення тендеру).

Взаємодія між клієнтським інтерфейсом та серверною частиною є повністю асинхронною і реалізована за допомогою вбудованого в браузер механізму Fetch API. Обмін даними відбувається виключно у фоновому режимі у форматі JSON, завдяки чому сторінки сайту не перезавантажуються повністю під час виконання операцій, забезпечуючи високу плавність роботи системи.

Для керування станом сесії користувача та його авторизацією в інтерфейсі застосовано глобальний об'єкт браузера localStorage. Після успішного виконання запиту на вхід, сервер повертає унікальний ідентифікатор користувача (userId) та його логін, які JavaScript зберігає у localStorage.

Цей механізм дозволяє реалізувати клієнтську логіку розмежування прав доступу. При завантаженні сторінки керування тендером скрипт зчитує userId із локального сховища і порівнює його з полем ownerId самого тендеру, який прийшов від API. Якщо ідентифікатори збігаються, JavaScript робить видимими елементи керування (кнопки “Видалити тендер”, “Змінити статус” або “Обрати переможця”). Якщо користувач є звичайним гостем або іншим виконавцем, ці елементи повністю приховуються або видаляються з розмітки сторінки, захищаючи інтерфейс від несанкціонованих дій.

Оформлення та адаптивність інтерфейсу реалізовано за допомогою каскадних таблиць стилів CSS. Використання Flexbox-моделей розмітки гарантує, що всі форми, таблиці та картки тендерів коректно відображаються як на моніторах персональних комп'ютерів, так і на екранах мобільних пристроїв.

ВИСНОВКИ

У курсовій роботі було успішно вирішено актуальне науково-практичне завдання із проектування, розробки та тестування сучасної інформаційної системи проведення тендерів. Створений програмний продукт автоматизує повний життєвий цикл взаємодії між замовниками та виконавцями: від публікації замовлення до збору комерційних пропозицій і фіксації переможця.

Під час виконання роботи було повністю досягнуто поставленої мети та вирішено такі ключові завдання:

Проаналізовано предметну область електронної комерції та замовлень, на основі чого сформовано детальні функціональні і нефункціональні вимоги до системи. Побудовано стандартизовану UML-діаграму прецедентів, яка чітко розмежувала ролі гостя та авторизованого користувача, а також детально розписано текстові сценарії для кожного прецеденту, що усунуло логічні суперечності на етапі проектування.

Спроектовано нормалізовану реляційну базу даних MySQL, яка складається з трьох взаємопов'язаних таблиць: `users`, `tenders` та `tender_proposals`. Завдяки впровадженню зовнішніх ключів, механізмів каскадного видалення та унікальних складених індексів на рівні СКБД, було забезпечено стовідсоткову цілісність даних та надійний захист від бізнес-помилки. Структуру бази даних зафіксовано у вигляді ER-діаграми.

Розроблено модульну серверну частину (Back-end) мовою Java на основі сучасної специфікації Jakarta RESTful Web Services (JAX-RS / Jersey). Реалізовано трирівневу архітектуру (Resource – Service – Model) та принципи REST API. Це дозволило повністю ізолювати бізнес-логіку системи від інтерфейсу, забезпечити жорстку валідацію вхідних даних, безпеку операцій та захист від SQL-ін'єкцій завдяки використанню параметризованих запитів `PreparedStatement` у технології JDBC. Архітектурні зв'язки між Java-класами наочно продемонстровано на UML-діаграмі класів системи.

Створено адаптивний клієнтський інтерфейс (Front-end) на базі технологій HTML5, CSS3 та чистого JavaScript без використання важковагових фреймворків. Завдяки інтеграції асинхронного механізму `Fetch API` було організовано ефективний обмін даними з сервером у фоновому режимі (формат JSON). Використання браузерного сховища `localStorage` дозволило надійно керувати станом сесії користувача та динамічно обмежувати доступ до елементів керування тендером (приховування кнопок видалення та вибору переможця для сторонніх осіб).

Розроблена інформаційна система має високу швидкість відгуку, інтуїтивно зрозумілий інтерфейс і повністю готова до розгортання в контейнерах сервлетів. Завдяки використанню архітектурного стилю REST API система є гнучкою та масштабованою, що дозволяє у майбутньому легко модернізувати її.

Перспективи подальшого розвитку проєкту полягають у розширенні функціоналу: впровадженні системи автоматичних email-сповіщень про нові пропозиції, додаванні рейтингу надійності для виконавців, реалізації розширеного пошуку з фільтрацією за категоріями та бюджетом, а також створенні окремого модуля адміністратора для модерації оголошень.

Завдання курсової роботи виконано в повному обсязі, а отримані результати підтверджено працездатністю створеного прототипу системи.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

Data Transfer Object [Електронний ресурс] - Режим доступу:

<https://uk.wikipedia.org/wiki/DTO>

Data Access Object [Електронний ресурс] - Режим доступу:

https://uk.wikipedia.org/wiki/Data_access_object

REST [Електронний ресурс] - Режим доступу:

<https://uk.wikipedia.org/wiki/REST>

Fetch API на MDN Web Docs [Електронний ресурс] - Режим доступу:

https://developer.mozilla.org/ru/docs/Web/API/Fetch_API/Using_Fetch

Sql-ін'єкція [Електронний ресурс] - Режим доступу:

<https://uk.wikipedia.org/wiki/SQL-%D1%96%D0%BD%27%D1%94%D0%BA%D1%86%D1%96%D1%8F>

ДОДАТКИ

webapp:

login.html

```

<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Вхід - Система Тендерів</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, sans-serif; background-color: #f4f6f9; display: flex;
      justify-content: center; align-items: center; height: 100vh; margin: 0; }
    .panel { background: white; padding: 30px; border-radius: 10px; box-shadow: 0 4px 15px rgba(0,0,0,0.1);
      width: 100%; max-width: 350px; }
    h2 { text-align: center; color: #2c3e50; margin-top: 0; }
    .form-group { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; font-weight: bold; font-size: 14px; }
    input { width: 100%; padding: 10px; border: 1px solid #ddd; border-radius: 5px; box-sizing: border-box; }
    button { background-color: #3498db; color: white; border: none; padding: 12px; font-size: 16px;
      border-radius: 5px; cursor: pointer; width: 100%; font-weight: bold; margin-top: 10px; }
    button:hover { background-color: #2980b9; }
    #loginError { color: red; display: none; font-size: 14px; text-align: center; margin-top: 10px; }
    p { margin-top: 25px; font-size: 14px; color: #7f8c8d; text-align: left; line-height: 1.5;
      display: flex; justify-content: center; }
    .switch-link { color: #2980b9; text-decoration: none; font-weight: 500; display: flex; justify-content: center; }
  </style>
</head>
<body>

<div class="panel">
  <h2>Вхід у систему</h2>
  <form id="loginForm">
    <div class="form-group">
      <label>Логін:</label>

```

```

        <input type="text" id="login" required>
    </div>
    <div class="form-group">
        <label>Пароль:</label>
        <input type="password" id="password" required>
    </div>
    <button type="submit">Увійти</button>
    <p id="loginError">Неправильний логін або пароль</p>
</form>

<p>
    Ще немає акаунту?<br>
    <a href="register.html" class="switch-link">Зареєструватися</a>
</p>
</div>

<script>
    document.getElementById('loginForm').addEventListener('submit', async function(event) {
        event.preventDefault();
        const credentials = {
            login: document.getElementById('login').value,
            password: document.getElementById('password').value
        };
        try {

```

```

            try {
                const response = await fetch('/api/users/login', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify(credentials)
                });

                if (response.ok) {
                    const user = await response.json();
                    localStorage.setItem('loggedUser', JSON.stringify(user));
                    window.location.href = 'index.html';
                } else {
                    document.getElementById('loginError').style.display = 'block';
                }
            } catch (error) {
                console.error('Помилка:', error);
            }
        });
    </script>
</body>
</html>

```

register.html

```

<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Регістрація</title>
  <style>
    body { background-color: #f4f6f9; display: flex; justify-content: center;
      align-items: center; height: 100px; margin: 0; }

    .login-card { background: white; padding: 40px 30px; border-radius: 8px;
      width: 100%; max-width: 350px; text-align: center; box-sizing: border-box; }

    .login-card h2 { color: #2c3e50; margin-top: 0; margin-bottom: 25px;
      font-size: 26px; font-weight: bold; }

    .input-group { text-align: left; margin-bottom: 18px; }

    .input-group label { display: block; font-weight: bold; margin-bottom: 6px; color: #000; font-size: 15px; }

    .input-group input { width: 100%; padding: 10px 12px; border: 1px solid #f3f3f3; border-radius: 5px;
      box-sizing: border-box; font-size: 15px; outline: none; }

    .input-group input:focus { border-color: #27ae60; }

    .btn-action { width: 100%; background-color: #27ae60; color: white; border: none;
      padding: 12px; border-radius: 5px; font-size: 16px; font-weight: bold; cursor: pointer;
      margin-top: 10px; }

    .btn-action:hover { background-color: #219653; }
  </style>

```



```

    </style>
</head>
<body>

<div class="login-card">
  <h2>Створення акаунту</h2>

  <div class="input-group">
    <label>Придумайте логін:</label>
    <input type="text" id="regLogin">
  </div>

  <div class="input-group">
    <label>Ваше ім'я:</label>
    <input type="text" id="regName">
  </div>

  <div class="input-group">
    <label>Придумайте пароль:</label>
    <input type="password" id="regPassword">
  </div>

  <button class="btn-action" onclick="register()">Створити акаунт</button>
</div>

<script>
  async function register() { Show usages & Newgra
    const loginVal = document.getElementById('regLogin').value.trim();
    const nameVal = document.getElementById('regName').value.trim();
    const passwordVal = document.getElementById('regPassword').value.trim();
  }
</script>

```

```

const nameVal = document.getElementById('regName').value.trim();
const passVal = document.getElementById('regPassword').value.trim();

if (!loginVal || !nameVal || !passVal) {
    alert("Будь ласка, заповніть всі поля!");
    return;
}

try {
    const response = await fetch('/api/users/register', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            login: loginVal,
            username: nameVal,
            password: passVal
        })
    });

    if (response.ok) {
        alert("Акаунт успішно створено! Тепер ви можете увійти.");
        window.location.href = 'login.html';
    } else {
        const err = await response.json();
        alert(err.error || "Помилка реєстрації. Можливо, такий логін вже існує.");
    }
}

```

```

    } catch (error) {
        console.error("Помилка реєстрації:", error);
        alert("Помилка з'єднання з сервером.");
    }
}

</script>
</body>
</html>

```

index.html

```

<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Тендери</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, sans-serif; background-color: #f4f6f9; padding: 20px; color: #333; margin: 0 }
    .container { max-width: 800px; margin: 0 auto; }
    .header-bar { display: flex; justify-content: space-between; align-items: center; background: white; padding: 15px 20px }
    h1 { color: #2c3e50; margin: 0; font-size: 24px; }
    button { border: none; padding: 10px 15px; font-size: 14px; border-radius: 5px; cursor: pointer; font-weight: bold; }
    .btn-logout { background-color: #e74c3c; color: white; }
    .btn-create { background-color: #3498db; color: white; display: none; margin-bottom: 20px; width: 100%; font-size: 14px; }
    .btn-apply { background-color: #27ae60; color: white; margin-top: 10px; }
    .tender-card { background: #fff; padding: 20px; margin-bottom: 15px; border-radius: 8px; border-left: 5px solid #3498db; }
    .tender-title { font-size: 20px; font-weight: bold; margin-bottom: 10px; }
    .tender-meta { font-size: 13px; color: #888; background: #ecf0f1; padding: 5px 10px; border-radius: 3px; display: inline-block; }
  </style>
</head>
<body>

<div class="container">
  <div class="header-bar">
    <h1>Тендери</h1>
    <div>
      <span id="userInfo" style="margin-right: 15px;"></span>
      <button class="btn-logout" onclick="logout()">Вийти</button>
    </div>
  </div>
</div>

```

```

<button id="btnCreateTender" class="btn-create" onclick="window.location.href='createTender.html'">
  + Створити новий тендер
</button>

<div style="display: flex; gap: 10px; margin-bottom: 20px;">
  <input type="text" id="searchInput" placeholder="Пошук за назвою або описом" style="flex-grow: 1; margin-bottom: 0;"
  <button onclick="performSearch()" style="background-color: #3498db; width: auto;">Знайти</button>
  <button onclick="loadTenders(); document.getElementById('searchInput').value='';" style="background-color: #95a5a6;"
</div>

<div id="tendersList"><p>Завантаження...</p></div>
</div>

<script>
  const user = JSON.parse(localStorage.getItem('loggedUser'));
  if (!user) window.location.href = 'login.html';

  if (user) {
    window.location.href = 'login.html';
  } else {
    document.getElementById('userInfo').innerText = user.username;
    document.getElementById('btnCreateTender').style.display = 'block';

    loadTenders();
  }

```

```

function logout() { Show usages & Newgra
  localStorage.removeItem('user'); // Змінили ключ тут
  window.location.href = 'login.html';
}

// Малює HTML з отриманого масиву
function renderTendersList(tenders) { Show usages & Newgra *
  const listContainer = document.getElementById('tendersList');
  listContainer.innerHTML = '';

  if (tenders.length === 0) {
    listContainer.innerHTML = '<p>За вашим запитом нічого не знайдено.</p>';
    return;
  }

  tenders.forEach(tender => {
    let actionBtn = '';
    if (user.id === tender.ownerId) {
      actionBtn = '<br><button style="background-color: #f39c12; color: white; border: none; padding: 8px 15px; border-radius: 5px; cursor: pointer; margin-top: 10px; font-weight: bold;"
        onclick="window.location.href=\'manageTender.html?id=${tender.id}\'">Керувати тендером</button>';
    } else {
      actionBtn = '<br><button style="background-color: #3498db; color: white; border: none; padding: 8px 15px; border-radius: 5px; cursor: pointer; margin-top: 10px; font-weight: bold;"
        onclick="window.location.href=\'tender.html?id=${tender.id}\'">Детальніше / Відгукнутись</button>';
    }
  }

```

```

const statusColor = (tender.status === 'CLOSED') ? '#e74c3c' : '#27ae60';

listContainer.innerHTML += `
  <div class="tender-card">
    <div class="tender-title">#${tender.id} - ${tender.name}</div>
    <div style="...">${tender.description}</div>
    <div class="tender-meta">Власник ID: ${tender.ownerId}</div>
    <div class="tender-meta" style="...">Статус: ${tender.status || 'ACTIVE'}</div>
    ${actionBtn}
  </div>
`;

});
}

async function loadTenders() { Show usages  Newgr
  try {
    const response = await fetch('/api/tenders');
    const tenders = await response.json();
    renderTendersList(tenders);
  } catch (error) {
    console.error("Помилка завантаження:", error);
  }
}

```

```

async function performSearch() { Show usages  Newgr
  const keyword = document.getElementById('searchInput').value;
  try {
    const response = await fetch(`/api/tenders/search?query=${encodeURIComponent(keyword)}`);
    const tenders = await response.json();
    renderTendersList(tenders);
  } catch (error) {
    console.error("Помилка пошуку:", error);
  }
}

// Пошук по натисканню Enter
document.getElementById('searchInput').addEventListener('keypress', function(event) {
  if (event.key === 'Enter') {
    event.preventDefault();
    performSearch();
  }
});
});
</script>
</body>
</html>

```

tender.html

```

<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Деталі тендери</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, sans-serif; background-color: #f4f6f9; padding: 20px; }
    .panel { background: white; padding: 30px; border-radius: 10px; max-width: 800px; margin: 20px auto; box-shadow: 0
    h2 { color: #2c3e50; margin-top: 0; }
    .info-block { background: #ecf0f1; padding: 15px; border-radius: 5px; margin-bottom: 20px; }

    .form-group { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; font-weight: bold; }
    input, textarea { width: 100%; padding: 10px; border: 1px solid #ddd; border-radius: 5px; box-sizing: border-box; }

    button { border: none; padding: 10px 15px; font-size: 14px; border-radius: 5px; cursor: pointer; font-weight: bold; }
    .btn-back { background-color: #95a5a6; color: white; margin-bottom: 15px; }
    .btn-submit { background-color: #27ae60; color: white; width: 100%; padding: 12px; font-size: 16px; }
  </style>
</head>
<body>

<div class="panel">
  <button class="btn-back" onclick="window.location.href='index.html'">Назад до списку</button>

  <div class="info-block" id="tenderInfo">
    <h2 id="tName">Завантаження...</h2>
    <p id="tDesc"></p>
    <p><strong>Статус:</strong> <span id="tStatus" style="color: #27ae60;"></span></p>
    <p><strong>Замовник:</strong> <span id="tOwner">Завантаження...</span></p>
  </div>

```

```

  </div>

  <div id="executorPanel" style="display: none; border-top: 2px solid #eee; padding-top: 20px;">
    <h3>Подати пропозицію</h3>
    <form id="proposalForm">
      <div class="form-group">
        <label>Ваша ціна (грн):</label>
        <input type="number" id="price" required min="1">
      </div>
      <div class="form-group">
        <label>Коментар (чому саме ви?):</label>
        <textarea id="proposalDesc" rows="3" required></textarea>
      </div>
      <button type="submit" class="btn-submit">Відправити заявку</button>
    </form>
  </div>
</div>

<script>
  const user = JSON.parse(localStorage.getItem('loggedUser'));
  if (!user) window.location.href = 'login.html';

  const urlParams = new URLSearchParams(window.location.search);
  const tenderId = urlParams.get('id');

  async function loadTenderDetails() { Show usages  Newgr *

```

```

async function loadTenderDetails() { Show usages  Newgr *
  if (!tenderId) {
    document.getElementById('tName').innerText = "Помилка: Тендер не знайдено";
    return;
  }

  try {
    const response = await fetch('/api/tenders/' + tenderId);
    if (response.ok) {
      const tender = await response.json();

      document.getElementById('tName').innerText = `#${tender.id} - ${tender.name}`;
      document.getElementById('tDesc').innerText = tender.description;
      document.getElementById('tStatus').innerText = tender.status;
      document.getElementById('tOwner').innerText = tender.ownerName || `ID ${tender.ownerId}`;

      if (user.id !== tender.ownerId) {
        if (tender.status === 'ACTIVE') {
          document.getElementById('executorPanel').style.display = 'block';
        } else {
          document.getElementById('executorPanel').style.display = 'block';
          document.getElementById('executorPanel').innerHTML = `
            <div style="background: #f8d7da; color: #721c24; padding: 15px; border-radius: 5px;
              text-align: center; font-weight: bold; border: 1px solid #f5c6cb; margin-top: 20px;">
                Прийм пропозицій завершено. Тендер закритий або вже має виконавця.
            </div>
          `;
        }
      }
    }
  }
}

```

```

    } else {
        document.getElementById('tName').innerText = "Тендер не існує або видалений";
    }
} catch (error) {
    console.error(error);
}
}

document.getElementById('proposalForm').addEventListener('submit', async function(event) {
    event.preventDefault();
    const currentStatus = document.getElementById('tStatus').innerText.trim();
    if (currentStatus !== 'ACTIVE') {
        alert("Помилка: Прийом пропозицій до цього тендеру вже завершено!");
        window.location.reload();
        return;
    }

    const proposal = {
        tenderId: parseInt(tenderId),
        executorId: user.id,
        price: parseFloat(document.getElementById('price').value),
        description: document.getElementById('proposalDesc').value
    };

```



```
    try {
      const response = await fetch('/api/proposals', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(proposal)
      });

      if (response.ok) {
        alert("Ваша заявка успішно відправлена!");
        window.location.href = 'index.html';
      } else {
        const errorResult = await response.json();
        alert(errorResult.error || "Помилка відправки.");
      }
    } catch (error) {
      console.error("Помилка зв'язку:", error);
      alert("Помилка з'єднання з сервером.");
    }
  });

  loadTenderDetails();
</script>
</body>
</html>
```

createTender.html

```

<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Створити тендер</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, sans-serif; background-color: #f4f6f9; padding: 20px; }
    .panel { background: white; padding: 30px; border-radius: 10px; max-width: 600px; margin: 40px auto; box-shadow: 0 4px 0 #ddd; }
    .form-group { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; font-weight: bold; }
    input, textarea { width: 100%; padding: 10px; border: 1px solid #ddd; border-radius: 5px; box-sizing: border-box; }
    button { background-color: #3498db; color: white; border: none; padding: 12px; font-size: 16px; border-radius: 5px; cursor: pointer; }
    .btn-cancel { background-color: #95a5a6; margin-bottom: 15px; }
  </style>
</head>
<body>

<div class="panel">
  <button class="btn-cancel" onclick="window.location.href='index.html'">Повернутися до списку</button>
  <h2>Створити новий тендер</h2>

  <form id="tenderForm">
    <div class="form-group">
      <label>Назва тендеру:</label>
      <input type="text" id="name" required>
    </div>
    <div class="form-group">
      <label>Опис:</label>
      <textarea id="description" rows="4" required>
    </div>
  </form>
</div>

```

```

        <button type="submit">Опублікувати тендер</button>
      </form>
</div>

<script>
  const user = JSON.parse(localStorage.getItem('loggedUser'));
  if (!user) window.location.href = 'login.html';

  document.getElementById('tenderForm').addEventListener('submit', async function(event) {
    event.preventDefault();

    const newTender = {
      name: document.getElementById('name').value,
      description: document.getElementById('description').value,
      ownerId: user.id,
      status: "ACTIVE"
    };

    const response = await fetch('/api/tenders', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(newTender)
    });

    if (response.ok) {

```

```

      if (response.ok) {
        alert('Тендер успішно створено!');
        window.location.href = 'index.html';
      }
    });
  </script>
</body>
</html>

```

manageTender.html

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Керування тендером</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, sans-serif; background-color: #f4f6f9; padding: 20px; }
    .panel { background: white; padding: 30px; border-radius: 10px; max-width: 800px; margin: 20px auto; }
    h2 { color: #2c3e50; margin-top: 0; }
    .info-block { background: #fff3cd; border-left: 5px solid #f39c12; padding: 15px; margin-bottom: 20px; border-radius: 10px; }
    button { border: none; padding: 10px 15px; font-size: 14px; border-radius: 5px; cursor: pointer; font-weight: bold; }
    .btn-back { background-color: #95a5a6; color: white; margin-bottom: 15px; }
    .dashboard-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; margin-top: 20px; }
    .dash-card { border: 1px solid #ddd; padding: 20px; border-radius: 8px; text-align: center; background: white; }
    .dash-card h3 { margin-top: 0; color: #2c3e50; }
    .btn-status { background-color: #e67e22; color: white; width: 100%; padding: 12px; margin-top: 10px; border: none; }
    .btn-action { background-color: #e74c3c; color: white; width: 100%; padding: 12px; margin-top: 10px; border: none; }
  </style>
</head>
<body>

<div class="panel">
  <button class="btn-back" onclick="window.location.href='index.html'">Повернутися на головну</button>

  <div class="info-block">
    <h2 id="tName">Завантаження...</h2>
    <p><strong>Поточний статус:</strong> <span id="tStatus" style="font-weight: bold;"></span></p>
  </div>

  <h2>Панель керування</h2>

  </div>

```

```

  <h2>Панель керування</h2>

  <div class="dashboard-grid">
    <div class="dash-card" style="border-color: #e67e22;">
      <h3>Статус тендеру</h3>
      <p style="color: #7f8c8d; font-size: 14px;">Зупинить тендер, якщо ви вже обрали виконавця або хочете припинити тендер</p>
      <button class="btn-status" onclick="changeStatus()">Змінити статус</button>
    </div>

    <div class="dash-card" style="border-color: #e74c3c;">
      <h3 style="color: #e74c3c;">Видалити тендер</h3>
      <p style="color: #7f8c8d; font-size: 14px;">Видалення тендеру назавжди зітре його з бази даних. Цю дію не можна скасувати</p>
      <button class="btn-action" onclick="deleteTender()">Видалити тендер</button>
    </div>
  </div>

  <div id="proposalsContainer" style="margin-top: 30px; padding: 25px; background: white; border-radius: 10px; border: 1px solid #ddd;">
    <h2 style="margin-top: 0; color: #2c3e50; border-bottom: 2px solid #f4f6f9; padding-bottom: 10px;">Отримані пропозиції</h2>
    <div id="proposalsList" style="margin-top: 15px;">
      <div class="proposal-item">
        <div class="proposal-header">
          <div class="proposal-title">Назва тендеру</div>
          <div class="proposal-status">Статус</div>
          <div class="proposal-date">Дата</div>
        </div>
        <div class="proposal-body">
          <div class="proposal-description">Опис тендеру</div>
          <div class="proposal-price">Ціна</div>
        </div>
      </div>
    </div>
  </div>
</div>

<script>
  const user = JSON.parse(localStorage.getItem('loggedUser'));
  if (!user) window.location.href = 'login.html';

```

```

let tenderId = new URLSearchParams(window.location.search).get('id');
let currentTender = null;

async function verifyAndLoad() { Show usages  Newgr
  if (!tenderId) {
    alert("Помилка: Тендер не знайдено");
    window.location.href = 'index.html';
    return;
  }

  try {
    const response = await fetch('/api/tenders/' + tenderId);
    if (response.ok) {
      const tender = await response.json();
      currentTender = tender;

      document.getElementById('tName').innerText = `Тендер #${tender.id}: ${tender.name}`;
      document.getElementById('tStatus').innerText = tender.status;

      if (user.id !== tender.ownerId) {
        alert("У вас немає прав для доступу до цієї сторінки!");
        window.location.href = 'index.html';
        return;
      }

      document.getElementById('tName').innerText = `Тендер #${tender.id}: ${tender.name}`;
      document.getElementById('tStatus').innerText = tender.status;

      document.getElementById('tStatus').innerText = tender.status;
      document.getElementById('tStatus').style.color = tender.status === 'ACTIVE' ? '#27ae60' : '#e74c3c';

      viewProposals();
    } else {
      alert("Тендер не існує");
      window.location.href = 'index.html';
    }
  } catch (error) {
    console.error(error);
  }
}

async function changeStatus() { Show usages  Newgr
  const currentStatus = document.getElementById('tStatus').innerText;
  const newStatus = (currentStatus === 'ACTIVE') ? 'CLOSED' : 'ACTIVE';
  const isConfirmed = confirm(`Ви впевнені, що хочете змінити статус на ${newStatus}?`);
  if (!isConfirmed) return;

  try {
    const response = await fetch(`/api/tenders/${tenderId}/status?newStatus=${newStatus}`, {
      method: 'PUT'
    });

    if (response.ok) {

```

```

    if (response.ok) {
      verifyAndLoad();
    } else {
      alert("Помилка при зміні статусу на сервері.");
    }
  } catch (error) {
    console.error(error);
  }
}

async function viewProposals() {
  try {
    const response = await fetch('/api/proposals/tender/' + tenderId);
    if (response.ok) {
      const proposals = await response.json();
      const list = document.getElementById('proposalsList');
      list.innerHTML = '';

      if (proposals.length === 0) {
        list.innerHTML = '<p style="color: #7f8c8d; font-style: italic;">На цей тендер немає затверджених';
        return;
      }

      const currentStatus = document.getElementById('tStatus').innerText.trim();
      const winnerId = currentTender ? currentTender.executorId : null;

```

```

    proposals.forEach(p => {
      let selectBtnHtml = '';
      let badgeHtml = '';
      let cardBorderColor = '#3498db';
      let cardBackground = '#f8fafc';

      if (winnerId && p.executorId === winnerId) {
        cardBorderColor = '#27ae60';
        cardBackground = '#f0fff4';
        badgeHtml = `
          <div style="background-color: #27ae60; color: white; padding: 6px 12px;
            font-weight: bold; border-radius: 4px; display: inline-block;
            margin-bottom: 12px; font-size: 13px; box-shadow: 0 2px 4px rgba(39,174,96,0.2);">
            ПЕРЕМОЖЕЦЬ
          </div>
        `;
      }

      if (currentStatus === 'ACTIVE') {
        selectBtnHtml = `
          <button onclick="chooseExecutor(${p.executorId})"
            style="margin-top: 15px; background-color: #27ae60; color: white;
            border: none; padding: 10px 15px; border-radius: 5px; cursor: pointer;
            font-weight: bold; font-size: 14px; width: 100%;">
            Обрати цього виконавця
          </button>
        `;
      }
    });

```

```

    `;
  }

  list.innerHTML += `
    <div style="...">
      ${badgeHtml}
      <div style="..."> Ціна: ${p.price} грн</div>
      <div style="..."> <strong>Коментар:</strong> ${p.description}</div>
      <div style="...">
        <strong>Виконавець:</strong> ${p.executorName || 'ID ' + p.executorId}
      </div>
      ${selectBtnHtml}
    </div>
  `;
});
}
} catch (error) {
  console.error("Помилка завантаження пропозицій:", error);
}
}

async function deleteTender() { Show usages  ⌕ Newgra
  const isConfirmed = confirm("УВАГА: Ви дійсно хочете назавжди видалити цей тендер та всі його заявки?");
  if (!isConfirmed) return;

```

```

    try {
      const response = await fetch(`/api/tenders/` + tenderId, {
        method: 'DELETE'
      });

      if (response.ok) {
        alert("Тендер успішно видалено!");
        window.location.href = 'index.html';
      } else {
        alert("Сталася помилка при видаленні.");
      }
    } catch (error) {
      console.error(error);
    }
  }

  async function chooseExecutor(executorId) { Show usages  ⌕ Newgra
    if (!confirm("Ви впевнені, що хочете обрати цього виконавця?")) {
      return;
    }

    try {
      const response = await fetch(`/api/tenders/${tenderId}/select-executor?executorId=${executorId}`, {
        method: 'PUT'
      });
    }
  }

```

```
    if (response.ok) {  
        alert("Виконавця успішно затверджено! Тендер закрито.");  
        location.reload();  
    } else {  
        alert("Сталася помилка при збереженні.");  
    }  
} catch (error) {  
    console.error(error);  
}  
}  
  
verifyAndLoad();  
</script>  
</body>  
</html>
```

model

Tender.java

```

public class Tender { 20 usages  🐞 Newgra *
    private int id; 2 usages
    private String name; 2 usages
    private String description; 2 usages
    private String status; 2 usages
    private int ownerId; 2 usages
    private String ownerName; 2 usages
    private int executorId; 2 usages

    public Tender(){} 3 usages  🐞 Newgra

    public int getId() { return id; }  🐞 Newgra
    public void setId(int id) { this.id = id; }  🐞 Newgra

    public String getName() { return name; } 1 usage  🐞 Newgra
    public void setName(String name) { this.name = name; } 3 usages  🐞 Newgra

    public String getDescription() { return description; } 1 usage  🐞 Newgra
    public void setDescription(String description) { this.description = description; } 3 usages  🐞 Newgra

    public String getStatus() { return status; }  🐞 Newgra
    public void setStatus(String status) { this.status = status; }  🐞 Newgra

    public int getOwnerId() { return ownerId; } 1 usage  🐞 Newgra
    public void setOwnerId(int ownerId) { this.ownerId = ownerId; } 3 usages  🐞 Newgra

    public String getOwnerName() { return ownerName; } no usages  🐞 Newgra
    public void setOwnerName(String ownerName) { this.ownerName = ownerName; } 3 usages  🐞 Newgra
    public int getExecutorId() { return executorId; } no usages  🐞 Newgra
    public void setExecutorId(int executorId) { this.executorId = executorId; } 2 usages

```

TenderProposal

```
package com.example.tendersystem.model;

public class TenderProposal { 9 usages  ⓘ Newgra
    private int id; 2 usages
    private int tenderId; 2 usages
    private int executorId; 2 usages
    private double price; 2 usages
    private String description; 2 usages

    public TenderProposal(){} 1 usage ⓘ Newgra

    public int getId() { return id; } ⓘ Newgra
    public void setId(int id) { this.id = id; } ⓘ Newgra

    public int getTenderId() { return tenderId; } 5 usages ⓘ Newgra
    public void setTenderId(int tenderId) { this.tenderId = tenderId; } 1 usage ⓘ Newgra

    public int getExecutorId() { return executorId; } 3 usages ⓘ Newgra
    public void setExecutorId(int executorId) { this.executorId = executorId; } 1 usage

    public double getPrice() { return price; } 1 usage ⓘ Newgra
    public void setPrice(double price) { this.price = price; } 1 usage ⓘ Newgra

    public String getDescription() { return description; } 1 usage ⓘ Newgra
    public void setDescription(String description) { this.description = description; }

}
```

User

```

package com.example.tendersystem.model;

public class User { 8 usages  ⚡ Newgra
    private int id; 2 usages
    private String username; 2 usages
    private String login; 2 usages
    private String password; 2 usages

    public User(){ 1 usage  ⚡ Newgra

    public int getId() { return id; }  ⚡ Newgra
    public void setId(int id) { this.id = id; }  ⚡ Newgra

    public String getUsername() { return username; } 1 usage  ⚡ Newgra
    public void setUsername(String username) { this.username = username; }

    public String getLogin() { return login; } 2 usages  ⚡ Newgra
    public void setLogin(String login) { this.login = login; } 1 usage  ⚡ Newgra

    public String getPassword() { return password; } 2 usages  ⚡ Newgra
    public void setPassword(String password) { this.password = password; }

}

```

resources

tenderResources

```
@Path("/tenders")
public class TenderResource {
    private final TenderService tenderService = new TenderService();

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<Tender> getAllTenders() { return tenderService.getAllTenders(); }

    @POST
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response createTender(Tender tend){
        Tender newTender = tenderService.createTender(tend);
        return Response.status(Response.Status.CREATED).entity(newTender).build();
    }

    @GET
    @Path("/{id}")
    @Produces(MediaType.APPLICATION_JSON)
    public Response getTenderById(@PathParam("id") int id) {
        Tender tender = tenderService.getTenderById(id);
        if (tender != null) {
            return Response.ok(tender).build();
        } else {
            return Response.status(Response.Status.NOT_FOUND).build();
        }
    }
}
```

```

@PUT @Newgra
@Path("/{id}/status")
@Consumes(MediaType.APPLICATION_JSON)
public Response changeStatus(@PathParam("id") int id, @QueryParam("newStatus") String newStatus) {
    boolean isUpdated = tenderService.updateTenderStatus(id, newStatus);

    if (isUpdated) {
        return Response.ok().build();
    } else {
        return Response.status(Response.Status.INTERNAL_SERVER_ERROR).build();
    }
}

@GET @Newgra
@Path("/{id}/search")
@Produces(MediaType.APPLICATION_JSON)
public Response searchTenders(@QueryParam("query") String query) {
    if (query == null || query.trim().isEmpty()) {
        return Response.ok(tenderService.getAllTenders()).build();
    }

    List<Tender> tenders = tenderService.searchTenders(query);
    return Response.ok(tenders).build();
}

```

```

@DELETE @Newgra
@Path("/{id}")
public Response deleteTender(@PathParam("id") int id) {
    boolean isDeleted = tenderService.deleteTender(id);

    if (isDeleted) {
        return Response.ok().build(); // Статус 200 OK
    } else {
        return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
            .entity("Помилка при видаленні тендеру")
            .build();
    }
}

@PUT @Newgra
@Path("/{id}/select-executor")
public Response selectExecutor(@PathParam("id") int tenderId, @QueryParam("executorId") int executorId) {
    boolean isUpdated = tenderService.selectExecutor(tenderId, executorId);

    if (isUpdated) {
        return Response.ok().build();
    } else {
        return Response.status(Response.Status.INTERNAL_SERVER_ERROR).entity("Помилка").build();
    }
}
}

```

ProposalResource

```
@Path(Ⓜ"/proposals") Ⓜ Newgra
public class ProposalResource {
    private final ProposalService proposalService = new ProposalService(); 2 usages

    @POST Ⓜ Newgra
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response applyForTender(TenderProposal proposal) {
        boolean isCreated = proposalService.createProposal(proposal);

        if (isCreated) {
            return Response.status(Response.Status.CREATED).build();
        } else {
            // Замість INTERNAL_SERVER_ERROR повертаємо BAD_REQUEST з описом помилки
            return Response.status(Response.Status.BAD_REQUEST)
                .entity(Ⓜ "{\\"error\\":\\"Не вдалося подати пропозицію. Можливо, ви вже подавали заявку або тендер закритий!\\"}")
                .build();
        }
    }

    @GET Ⓜ Newgra
    @Path(Ⓜ"/tender/{tenderId}")
    @Produces(MediaType.APPLICATION_JSON)
    public Response getProposals(@PathParam("tenderId") int tenderId) {
        java.util.List<TenderProposal> proposals = proposalService.getProposalsByTenderId(tenderId);
        return Response.ok(proposals).build();
    }
}
```

UserResource

```
@Path(Ⓜ"/users") Ⓜ Newgra
public class UserResource {
    private final UserService userService = new UserService(); 2 usages

    @POST Ⓜ Newgra
    @Path(Ⓜ"/login")
    @Consumes(MediaType.APPLICATION_JSON)
    @Produces(MediaType.APPLICATION_JSON)
    public Response login(User loginData) {
        // Шукаємо користувача в базі
        User dbUser = userService.authenticateUser(loginData.getLogin(), loginData.getPassword());

        if (dbUser != null) {
            // ОБОВ'ЯЗКОВО: відправляємо знайденого користувача назад (dbUser)
            return Response.ok(dbUser).build();
        } else {
            return Response.status(Response.Status.UNAUTHORIZED)
                .entity(Ⓜ "{\\"error\\":\\"Невірний логін або пароль!\\"}")
                .build();
        }
    }
}
```

```

@POST @Newgra
@Path("/register")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response register(User user) {
    boolean success = userService.registerUser(user.getUsername(), user.getLogin(), user.getPassword());

    if (success) {
        return Response.ok(entity("{\"message\": \"Успішна реєстрація!\"}").build());
    } else {
        return Response.status(Response.Status.BAD_REQUEST)
            .entity(entity("{\"error\": \"Помилка! Можливо, такий логін вже зайнятий.\"}"))
            .build();
    }
}
}

```

service

```

public class TenderService { 3 usages @Newgra
    private static final String DB_URL = "jdbc:mysql://localhost:3306/tender_db"; 7 usages
    private static final String USER = "root"; 7 usages
    private static final String PASS = System.getenv("DB_PASSWORD"); 7 usages

    public List<Tender> getAllTenders() { 2 usages @Newgra
        List<Tender> list = new ArrayList<>();
        try { Class.forName("com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { return list; }

        // LEFT JOIN гарантує, що тендер виведеться, навіть якщо юзера випадково видалили
        String sql = "SELECT t.*, u.username AS ownerName FROM tenders t LEFT JOIN users u ON t.ownerId = u.id";

        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                Tender tender = new Tender();
                tender.setId(rs.getInt("id"));
                tender.setName(rs.getString("name"));
                tender.setDescription(rs.getString("description"));
                tender.setStatus(rs.getString("status"));
                tender.setOwnerId(rs.getInt("ownerId"));
                tender.setOwnerName(rs.getString("ownerName"));
                list.add(tender);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

```

        return list;
    }

    public Tender createTender(Tender tend) { 1 usage 2 Newgra
        try { Class.forName("com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { return null; }
        String sql = "INSERT INTO tenders (name, description, status, ownerId) VALUES (?, ?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
            PreparedStatement pstmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

            pstmt.setString(parameterIndex: 1, tend.getName());
            pstmt.setString(parameterIndex: 2, tend.getDescription());
            pstmt.setString(parameterIndex: 3, tend.getStatus());
            pstmt.setInt(parameterIndex: 4, tend.getOwnerId());

            int affectedRows = pstmt.executeUpdate();

            if (affectedRows == 0) {
                throw new SQLException("Створення тендеру не вдалося, жодного рядка не змінено.");
            }

            try (ResultSet generatedKeys = pstmt.getGeneratedKeys()) {
                if (generatedKeys.next()) {
                    tend.setId(generatedKeys.getInt(columnIndex: 1));
                } else {
                    throw new SQLException("Створення тендеру не вдалося, ID не отримано.");
                }
            }
        }
    }

```

```

    } catch (SQLException e) {
        System.err.println("Помилка під час збереження в базу:");
        e.printStackTrace();
        throw new RuntimeException("Не вдалося зберегти тендер у БД", e);
    }

    return tend;
}

    public Tender getTenderById(int id) { 1 usage 2 Newgra
        try { Class.forName("com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { return null; }

        String sql = "SELECT t.*, u.username AS ownerName FROM tenders t LEFT JOIN users u ON t.ownerId = u.id WHERE t.id = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(parameterIndex: 1, id);

            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    Tender t = new Tender();
                    t.setId(rs.getInt(columnLabel: "id"));
                    t.setName(rs.getString(columnLabel: "name"));
                    t.setDescription(rs.getString(columnLabel: "description"));
                    t.setStatus(rs.getString(columnLabel: "status"));
                    t.setOwnerId(rs.getInt(columnLabel: "ownerId"));
                    t.setOwnerName(rs.getString(columnLabel: "ownerName"));
                }
            }
        }
    }

```



```

        t.setExecutorId(rs.getInt( columnLabel: "executorId"));

        return t;
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return null;
}

public boolean updateTenderStatus(int tenderId, String newStatus) { 1 usage 2 Newgra
    try { Class.forName( className: "com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { return false; }

    String sql;
    if ("ACTIVE".equalsIgnoreCase(newStatus)) {
        sql = "UPDATE tenders SET status = ?, executorId = NULL WHERE id = ?";
    } else {
        sql = "UPDATE tenders SET status = ? WHERE id = ?";
    }

    try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString( parameterIndex: 1, newStatus);
        pstmt.setInt( parameterIndex: 2, tenderId);

        return pstmt.executeUpdate() > 0;
    }
}

```

```

    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public List<Tender> searchTenders(String keyword) { 1 usage 2 Newgra
    List<Tender> list = new ArrayList<>();
    try { Class.forName( className: "com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { return list; }

    String sql = "SELECT t.*, u.username AS ownerName FROM tenders t " +
        "LEFT JOIN users u ON t.ownerId = u.id " +
        "WHERE t.name LIKE ? OR t.description LIKE ?";

    try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        String searchPattern = "%" + keyword + "%";
        pstmt.setString( parameterIndex: 1, searchPattern);
        pstmt.setString( parameterIndex: 2, searchPattern);

        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Tender tender = new Tender();
                tender.setId(rs.getInt( columnLabel: "id"));
                tender.setName(rs.getString( columnLabel: "name"));
                tender.setDescription(rs.getString( columnLabel: "description"));
                tender.setStatus(rs.getString( columnLabel: "status"));
            }
        }
    }
}

```

```

        tender.setOwnerId(rs.getInt( columnLabel: "ownerId"));
        tender.setOwnerName(rs.getString( columnLabel: "ownerName"));
        tender.setExecutorId(rs.getInt( columnLabel: "executorId"));

        list.add(tender);
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return list;
}

public boolean deleteTender(int id) { 1 usage 2 Newgra
    try { Class.forName( className: "com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { return false; }

    String sqlDeleteProposals = "DELETE FROM tender_proposals WHERE tenderId = ?";
    String sqlDeleteTender = "DELETE FROM tenders WHERE id = ?";

    try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS)) {
        try (PreparedStatement pstmt1 = conn.prepareStatement(sqlDeleteProposals)) {
            pstmt1.setInt( parameterIndex: 1, id);
            pstmt1.executeUpdate();
        }

        try (PreparedStatement pstmt2 = conn.prepareStatement(sqlDeleteTender)) {
            pstmt2.setInt( parameterIndex: 1, id);
            int affectedRows = pstmt2.executeUpdate();

```

```

        return affectedRows > 0;
    }

} catch (SQLException e) {
    e.printStackTrace();
    return false;
}

}

public boolean selectExecutor(int tenderId, int executorId) { 1 usage 2 Newgra
    try { Class.forName( className: "com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { return false; }

    try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS)) {

        String sqlUpdateTender = "UPDATE tenders SET status = 'CLOSED', executorId = ? WHERE id = ?";
        try (PreparedStatement pstmt1 = conn.prepareStatement(sqlUpdateTender)) {
            pstmt1.setInt( parameterIndex: 1, executorId);
            pstmt1.setInt( parameterIndex: 2, tenderId);
            pstmt1.executeUpdate();
        }

        return true;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }

}

}

```

ProposalService

```

public class ProposalService { 3 usages 2 Newgra
    private static final String DB_URL = "jdbc:mysql://localhost:3306/tender_db"; 2 usages
    private static final String USER = "root"; 2 usages
    private static final String PASS = System.getenv( name: "DB_PASSWORD"); 2 usages

    public boolean createProposal(TenderProposal proposal) { 1 usage 2 Newgra
        try {
            Class.forName( className: "com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            return false;
        }

        String checkStatusSql = "SELECT status FROM tenders WHERE id = ?";
        String checkDuplicateSql = "SELECT COUNT(*) FROM tender_proposals WHERE tenderId = ? AND executorId = ?";
        String insertSql = "INSERT INTO tender_proposals (tenderId, executorId, price, description) VALUES (?, ?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS)) {
            try (PreparedStatement checkStmt = conn.prepareStatement(checkStatusSql)) {
                checkStmt.setInt( parameterIndex: 1, proposal.getTenderId());
                try (ResultSet rs = checkStmt.executeQuery()) {
                    if (rs.next()) {
                        String status = rs.getString( columnLabel: "status");
                        if (!"ACTIVE".equalsIgnoreCase(status)) {
                            System.out.println("Спроба зламу: тендер " + proposal.getTenderId() + " вже закритий!");
                            return false;
                        }
                    } else {
                        return false;
                    }
                }
            }
        }

        try (PreparedStatement checkDupStmt = conn.prepareStatement(checkDuplicateSql)) {
            checkDupStmt.setInt( parameterIndex: 1, proposal.getTenderId());
            checkDupStmt.setInt( parameterIndex: 2, proposal.getExecutorId());
            try (ResultSet rs = checkDupStmt.executeQuery()) {
                if (rs.next() && rs.getInt( columnIndex: 1) > 0) {
                    System.out.println("Відхилено: виконавець " + proposal.getExecutorId() + " вже подав заявку на тендер " + proposal.getTenderId());
                    return false;
                }
            }
        }

        try (PreparedStatement pstmt = conn.prepareStatement(insertSql)) {
            pstmt.setInt( parameterIndex: 1, proposal.getTenderId());
            pstmt.setInt( parameterIndex: 2, proposal.getExecutorId());
            pstmt.setDouble( parameterIndex: 3, proposal.getPrice());
            pstmt.setString( parameterIndex: 4, proposal.getDescription());

            int affectedRows = pstmt.executeUpdate();
            return affectedRows > 0;
        }

        tch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}

```

```

public List<TenderProposal> getProposalsByTenderId(int tenderId) { 1 usage 2 Newgra
    List<TenderProposal> list = new ArrayList<>();
    try { Class.forName( className: "com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { return list; }

    String sql = "SELECT * FROM tender_proposals WHERE tenderId = ?";

    try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt( parameterIndex: 1, tenderId);

        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                TenderProposal p = new TenderProposal();
                p.setId(rs.getInt( columnLabel: "id"));
                p.setTenderId(rs.getInt( columnLabel: "tenderId"));
                p.setExecutorId(rs.getInt( columnLabel: "executorId"));
                p.setPrice(rs.getDouble( columnLabel: "price"));
                p.setDescription(rs.getString( columnLabel: "description"));
                list.add(p);
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return list;
}
}

```

UserService.java

```

public class UserService { 3 usages 2 Newgra
    private static final String DB_URL = "jdbc:mysql://localhost:3306/tender_db";
    private static final String USER = "root"; 2 usages
    private static final String PASS = System.getenv( name: "DB_PASSWORD"); 2 usage

    public User authenticateUser(String login, String password) { 1 usage 2 Newgra
        try {
            Class.forName( className: "com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
            return null;
        }

        String sql = "SELECT * FROM users WHERE login = ? AND password = ?";

        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString( parameterIndex: 1, login);
            pstmt.setString( parameterIndex: 2, password);

            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    User user = new User();
                    user.setId(rs.getInt( columnLabel: "id"));
                    user.setUsername(rs.getString( columnLabel: "username"));
                    user.setLogin(rs.getString( columnLabel: "login"));
                    return user;
                }
            }
        }
    }
}

```

```

    } catch (SQLException e) {
        e.printStackTrace();
    }
    return null;
}

public boolean registerUser(String username, String login, String password) {
    try { Class.forName("com.mysql.cj.jdbc.Driver"); } catch (ClassNotFoundException e) { return false; }

    String sql = "INSERT INTO users (username, login, password) VALUES (?, ?, ?)";

    try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, username);
        pstmt.setString(2, login);
        pstmt.setString(3, password);

        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
}

```

MySQL – вигляд бази даних

tenders

	id	name	description	status	ownerId	executorId
▶	5	1 км дороги в Чорнобилі)	Потрібні потужні люди, які не жаліються на ...	ACTIVE	3	NULL
	17	Поставка 3000 крисЛіхтариків	Необхідно створити 3000 крисЛіхтариків і до...	ACTIVE	4	NULL
	18	Розробка корпоративного вебсайту	Створення адаптивного сайту для логістичн...	ACTIVE	1	NULL
	19	Постачання серверного обладнання	Закупівля та встановлення двох серверів De...	CLOSED	2	3
	20	Налаштування CI/CD пайплайну	Шукаємо DevOps-інженера для міграції проєк...	ACTIVE	4	NULL
	21	Розробка мобільного додатку (Flutter)	Створення кросплатформного MVP додатку ...	CLOSED	1	4
	22	SEO-оптимізація інтернет-магазину	Проведення глибокого технічного аудиту са...	ACTIVE	3	NULL
	23	Оновлення парку комп'ютерної техніки	Закупівля 20 ноутбуків для відділу розробки...	ACTIVE	2	NULL
•	NULL	NULL	NULL	NULL	NULL	NULL

tender_proposals

	id	tenderId	executorId	price	description
▶	18	17	2	45000.00	Готові виконати проєкт за 3 тижні. Включен...
	19	17	3	38500.50	Маємо великий досвід розробки подібних сай...
	20	18	4	125000.00	Пропонуємо обладнання зі складу в Києві. Га...
	21	19	2	15000.00	Маю 4 роки досвіду роботи з GitLab CI та Do...
	22	20	1	85000.00	Команда з 3 розробників. Працюємо на Flutte...
•	NULL	NULL	NULL	NULL	NULL

user

	id	username	login	password
▶	1	Kris	Kris322	SercretPassword
	2	Sergi	Sergi123	Password123
	3	ТОВ ТехАвангард	tech_avangard	SecurePass2026
	4	Макс	m_krishka	Krishka!51